

Data Quality Dimensions

1. Load Dataset

```
#Importing libraries and loading dataset
import pandas as pd
data = pd.read_excel('Assignment_11_SD_Dataset.xlsx')

#Creating a copy to work on and validating
df = data.copy()
df
```

	ID	NAME	AGE	EMAIL	DATE OF BIRTH	SALARY	YEARS OF SERVICE	DEPARTMENT
0	1	John Doe	29.0	john.doe@example.com	1995-02-10 00:00:00	50000.0	5.0	Sales
1	2	Jane Smith	NaN	Jane.smith@company.com	1980-07-25 00:00:00	NaN	4.0	HR
2	3	Mike Brown	35.0	mike.brown@example	03/12/1989	70000.0	6.0	IT
3	4	Emily Davis	22.0	emily.d@domain.com	2026-12-05 00:00:00	55000.0	5.0	Marketing
4	5	Chris Lee	46.0	NaN	1978-11-05 00:00:00	80000.0	6.0	Sales
5	6	Linda Black	NaN	linda.black@example.com	06-15-1993	56000.0	4.0	Finance
6	7	Matt Richard	29.0	NaN	1995-02-10 00:00:00	NaN	NaN	Sales
7	8	Sarah Blue	30.0	sarahblue@work.com	2026-03-05 00:00:00	60000.0	4.0	HR
8	9	NaN	42.0	jack.white@example	08/21/1982	120000.0	7.0	IT
9	10	Anna Green	27.0	anna.green@organization	1997-12-30 00:00:00	100000.0	6.0	Marketing

2. Identifying the issues with the dataset

```
#Getting an overview of the entire dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   ID                     10 non-null    int64
1   NAME                   9 non-null     object
2   AGE                    8 non-null     float64
3   EMAIL                  8 non-null     object
4   DATE OF BIRTH          10 non-null    object
5   SALARY                 8 non-null     float64
6   YEARS OF SERVICE       9 non-null     float64
7   DEPARTMENT             10 non-null    object
dtypes: float64(3), int64(1), object(4)
memory usage: 772.0+ bytes
```

```
df.describe()
```

	ID	AGE	SALARY	YEARS OF SERVICE
count	10.00000	8.000000	8.000000	9.000000
mean	5.50000	32.500000	73875.000000	5.222222
std	3.02765	8.017837	24787.309541	1.092906
min	1.00000	22.000000	50000.000000	4.000000
25%	3.25000	28.500000	55750.000000	4.000000
50%	5.50000	29.500000	65000.000000	5.000000
75%	7.75000	36.750000	85000.000000	6.000000
max	10.00000	46.000000	120000.000000	7.000000

```
#Checking for missing data
df.isnull().sum()
```

	0
ID	0
NAME	1
AGE	2
EMAIL	2
DATE OF BIRTH	0
SALARY	2
YEARS OF SERVICE	1
DEPARTMENT	0

dtype: int64

```
# Checking for duplicates
df.duplicated().sum()
```

np.int64(0)

```
#Checking value counts for department
print(df['DEPARTMENT'].nunique())
df['DEPARTMENT'].value_counts()
```

5	
	count
DEPARTMENT	
Sales	3
HR	2
IT	2
Marketing	2
Finance	1

dtype: int64

Problems this data has:

1. Missing values
2. Wrong data types
3. Email/Date formats are inconsistent
4. Column name format

3. Correcting and validating

```
# Changing the column names from uppercase to lowercase
df.columns = df.columns.str.lower()
```

```
#Checking
df.head(1)
```

	id	name	age	email	date of birth	salary	years of service	department
0	1	John Doe	29.0	john.doe@example.com	1995-02-10 00:00:00	50000.0	5.0	Sales

```
# Formatting the date column and changing the data type
```

```
#importing libraries needed
from dateutil.parser import parse
```

```
# Choosing the date of birth column, (apply)- treats every value individually
# (lambda(x)- creates a temporary function where x represents each cell one at a time
# (str(x)) converts everything to string
# (replace)- removes the time
df['date of birth'] = df['date of birth'].apply(lambda x: str(x).replace(' 00:00:00', ''))
```

```
# parse taking the values and seperating them to a uniform date format
# (if x != '') parsing on cells that contain value
# (else pd.NaT) if the cell is empty then store pd.NaT
df['date of birth'] = df['date of birth'].apply(
    lambda x: parse(x) if x != '' else pd.NaT)
```

```
# formatting the date format to Y/M/D
df['date of birth'] = df['date of birth'].dt.strftime('%Y-%m-%d')
```

```
#converting data type to date
df['date of birth'] = pd.to_datetime(df['date of birth'], errors='coerce')
```

```
#Viewing results/Checking data type
df['date of birth']
```

	date of birth
0	1995-02-10
1	1980-07-25
2	1989-03-12
3	2026-12-05
4	1978-11-05
5	1993-06-15
6	1995-02-10
7	2026-03-05
8	1982-08-21
9	1997-12-30

```
dtype: datetime64[ns]
```

```
# I have noticed invalid birth years (2026)!
# I will check for invalid birth years and correct them with a calculation from the age column
```

```
#Importing libraries needed
from datetime import datetime
```

```
#Storing the current year
current_year = datetime.now().year
```

```
# Calculating expected ages
df['calculated age'] = current_year - df['date of birth'].dt.year
```

```
#Looking for invalid d.o.b. years (adding one incase their birthday hasn't happened yet)
invalid_birth_year = abs(df['calculated age'] - df['age']) > 1
```

```
# Replacing the incorrect birth years
df.loc[invalid_birth_year, 'date of birth'] = df.loc[invalid_birth_year].apply(
    lambda row: row['date of birth'].replace(year=current_year - int(row['age'])), axis=1)
```

```
# Checking the results
df[['date of birth', 'age']]
```

	date of birth	age
0	1997-02-10	29
1	1980-07-25	45
2	1991-03-12	35
3	2004-12-05	22
4	1980-11-05	46
5	1993-06-15	32
6	1997-02-10	29
7	1996-03-05	30
8	1984-08-21	42
9	1999-12-30	27

```
#Dropping the calculated age column I created
df = df.drop(columns=['calculated age'])
```

```
#Filling missing values for age column

#Storing current date
today = pd.Timestamp.today()

#Creating a formula to calculate the age from date of birth column
calc_age = (today.year - df['date of birth'].dt.year -
            ((today.month < df['date of birth'].dt.month) |
             ((today.month == df['date of birth'].dt.month) &
              (today.day < df['date of birth'].dt.day))))

#Filling in the missing ages from calculation
df['age'] = df['age'].fillna(calc_age)

#Changing the data type
df['age'] = df['age'].astype(int)

#Checking the fill worked
display(df[['date of birth', 'age']])

#Checking the data conversion worked
df['age'].dtypes

#Checking for missing values
print('Number of missing values now:', df['age'].isnull().sum())
```

	date of birth	age
0	1997-02-10	29
1	1980-07-25	45
2	1991-03-12	35
3	2004-12-05	22
4	1980-11-05	46
5	1993-06-15	32
6	1997-02-10	29
7	1996-03-05	30
8	1984-08-21	42
9	1999-12-30	27

Number of missing values now: 0

```
#With this being a small dataset alot of the emails are just missing the .com portion.
#For this scenario I will add .com to these emails then check for further validation

df['email'] = df['email'].apply(lambda x: x + '.com' if pd.notna(x)
and '@' in str(x) and '.com' not in str(x) else x)

# This is saying, in our email column add .com if the value is not null, contains @
# and does not already contain .com

#Checking in worked
df['email']
```

	email
0	john.doe@example.com
1	Jane.smith@company.com
2	mike.brown@example.com
3	emily.d@domain.com
4	NaN
5	linda.black@example.com
6	NaN
7	sarahblue@work.com
8	jack.white@example.com
9	anna.green@organization.com

dtype: object

```
#Checking for valid emails

#importing the libraries needed
import re

#creating a function with regular expressions to check if email addresses are valid

def valid_email(email):
    if pd.isna(email):
        return False
    return bool(re.match(r"^[^@]+@[^@]+\.[^@]+$", str(email)))

# This function is saying check emails that are not missing, and match them to
# the format of string-@symbol-string-period-string(extension)

#Calling the function and replace my invalid emails with NA
df['email'] = df['email'].apply(lambda x: x if valid_email(x) else pd.NA)

#Checking the work
df['email']
```

	email
0	john.doe@example.com
1	Jane.smith@company.com
2	mike.brown@example.com
3	emily.d@domain.com
4	<NA>
5	linda.black@example.com
6	<NA>
7	sarahblue@work.com
8	jack.white@example.com
9	anna.green@organization.com

dtype: object

```
# Filling in the missing values of the salary column with a hierarchical strategy

# Grouping by department and years of service
#Filling the missing values with the average of each group that matches missing values

df['salary'] = df.groupby(['department', 'years of service'])
['salary'].transform(lambda x: x.fillna(x.mean()))

#Filling any missing values that do not fall into the grouping
df['salary'] = df['salary'].fillna(df['salary'].median())

#Checking missing values
df['salary'].isna().sum()
```

```
np.int64(0)
```

```
# On this small dataset, I can see that everyone's name matches their email so I will extract
# the missing names from the emails

df['name'] = df.apply(lambda row: row['email'].split('@')[0].replace('.', ' ').title()
if pd.isna(row['name']) and pd.notna(row['email']) else row['name'], axis=1)

# What this is saying: take our name column and email column, and apply this rule one row at a time.
# split the name portion from the email and return 0 index portion(name), turn the name into 'Title'
# then fill with the name if both conditions are true: name is missing and email exists, else leave name alone.

#Checking it worked
print(df['name'].isnull().sum())
df['name']
```

```
0
      name
0  John Doe
1  Jane Smith
2  Mike Brown
3  Emily Davis
4   Chris Lee
5  Linda Black
6  Matt Richard
7   Sarah Blue
8   Jack White
9   Anna Green
```

```
dtype: object
```

```
#Filling in years of service

#Checking the skew of my years of service column
print('The skew before:', df['years of service'].skew())

#Wanted to check skew, if it was only mildly skewed I thought average would work best for filling
df['years of service'] = df['years of service'].fillna(df['years of service'].mean())

#Checking the results
print('The null values:', df['years of service'].isnull().sum())

#Rechecking skew to ensure it is similar
print('The skew after:', df['years of service'].skew())
```

```
The skew before: 0.18846999389943142
The null values: 0
The skew after: 0.19434976565735965
```

```
# Fixing the data type for years of service
df['years of service'] = df['years of service'].astype(int)
```

```
#Checking
df['years of service'].dtypes

dtype('int64')
```

4. Summary and statistic reports (please see below)

5. Asses data completeness and quality

```
# Recheck the entire dataset
df
```

	id	name	age	email	date of birth	salary	years of service	department
0	1	John Doe	29	john.doe@example.com	1997-02-10	50000.0	5	Sales
1	2	Jane Smith	45	Jane.smith@company.com	1980-07-25	60000.0	4	HR
2	3	Mike Brown	35	mike.brown@example.com	1991-03-12	70000.0	6	IT
3	4	Emily Davis	22	emily.d@domain.com	2004-12-05	55000.0	5	Marketing
4	5	Chris Lee	46	<NA>	1980-11-05	80000.0	6	Sales
5	6	Linda Black	32	linda.black@example.com	1993-06-15	56000.0	4	Finance
6	7	Matt Richard	29	<NA>	1997-02-10	60000.0	5	Sales
7	8	Sarah Blue	30	sarahblue@work.com	1996-03-05	60000.0	4	HR
8	9	Jack White	42	jack.white@example.com	1984-08-21	120000.0	7	IT
9	10	Anna Green	27	anna.green@organization.com	1999-12-30	100000.0	6	Marketing

```
#Rechecking o=info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     10 non-null    int64
1   name                   10 non-null    object
2   age                    10 non-null    int64
3   email                  8 non-null     object
4   date of birth          10 non-null    datetime64[ns]
5   salary                 10 non-null    float64
6   years of service       10 non-null    int64
7   department             10 non-null    object
dtypes: datetime64[ns](1), float64(1), int64(3), object(3)
memory usage: 772.0+ bytes
```